

Sztringek C++-ban: az `std::string`

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ugyanúgy, mint a vektoros jegyzettel: olvass, próbáld ki, aztán oldd meg a feladatokat.

- **Próbáld ki** – gépeld be a kódot, és futtasd. Gépeld, ne másold: a kezed is tanul.
- **Tippelj** – előbb írd le a füzetedbe, szerinted mit ír ki a program, és csak utána futtasd.
- **Feladat** – ezt már te írod meg. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Néhány tudnivaló, mielőtt belevágsz:

- Minden programba kell az `#include <string>` sor az `#include <iostream>` mellé.
- A rövid kódrészleteket a `main` belsejébe írd.
- A szövegekbe ne írd ékezetes betűt. Hogy miért, az a 4. fejezetben kiderül.
- Jó hír: amit a vektoroknál megtanultál (indexelés, bejárás, megszámlálás, keresés), az itt is működik.

1. Szöveg egy változóban

A sztring (`std::string`) szöveget tárol: betűket, számjegyeket, szóközöket, írásjeleket.

Próbáld ki

```
#include <iostream>
#include <string>

int main() {
    std::string nev = "Anna";

    std::cout << "Szia, " << nev << "!" << std::endl;
    return 0;
}
```

A szöveget mindig dupla idézőjelek közé írjuk. Írd át a nevet a sajátodra, és futtasd újra.

Próbáld ki - összefűzés

```
std::string elso = "alma";
std::string masodik = "fa";
std::string egyutt = elso + masodik;

std::cout << egyutt << std::endl;
```

A `+` jel két sztringet egymás mögé ragaszt.

Tippelj

```
std::string a = "12";
std::string b = "34";
std::cout << a + b << std::endl;

std::string szo = "ha";
szo += "ha";
szo += "!";
std::cout << szo << std::endl;
```

Az első kiírásnál 46-ot vártál? Fogalmazd meg a füzetedben, miért nem azt kaptad.

Figyelem! Két idézőjeles szöveget közvetlenül nem lehet összeadni: az `"alma" + "fa"` fordítási hibát ad. Legalább az egyik oldalon sztring változó álljon.

Próbáld ki - a szöveg hossza

```
std::string szo = "informatika";

std::cout << szo.length() << std::endl;
```

A vektornál ezt `size()`-nak hívtuk. A sztringnél a `length()` és a `size()` ugyanazt jelenti.

Tippelj - mi számít bele a hosszba?

Mekkora a hossza ennek a három sztringnek? Írd le a tippedet, aztán ellenőrizd.

```
std::string a = "";
std::string b = " ";
std::string c = "Kiss Anna";
```

Feladatok

1.1. feladat ★☆☆ - Köszönés

Tárold el a keresztnedet egy sztringben. Írasd ki vele ezt a sort: `Szia, Anna! Udv a szakkoron.`

1.2. feladat ★☆☆ - Teljes név

Legyen két változód: `vezeteknev` és `keresztnev`. Fűzd össze őket egy harmadik, `teljes` nevű sztringbe úgy, hogy szóköz legyen közöttük. Írd ki a `teljes` értékét.

1.3. feladat ★☆☆ - Milyen hosszú?

Írd ki a teljes név hosszát. Számold meg kézzel is: stimmel? Mit számolt bele a program a betűkön kívül?

1.4. feladat ★☆☆ - Nevetés

Indulj egy üres sztringből. Ciklussal fűzd hozzá ötször a `"ha"` szöveget, majd írd ki az eredményt és a hosszát.

1.5. feladat ★☆☆ - Aláhúzás

Írd ki az `informatika` szót, alá pedig egy új sorba pontosan annyi kötőjelet, ahány betűből a szó áll. Írd át a szót másra: a kötőjelek száma magától igazodjon.

2. Beolvasás: szó vagy sor?

A `std::cin >>` egyetlen szót olvas be. Egy egész sort a `std::getline` olvas be.

Próbáld ki

```
#include <iostream>
#include <string>

int main() {
    std::string nev;
    std::cout << "Mi a neved? ";
    std::cin >> nev;

    std::cout << "Szia, " << nev << "!" << std::endl;
    return 0;
}
```

Futtasd kétszer. Először csak a keresztnévedet írd be, másodszor a teljes nevedet. Mit köszön vissza a program a második esetben?

Amit megfigyelhettél: a `std::cin >>` az első szóköznel megáll.

Próbáld ki - két szó

```
std::string vezeteknev, keresztnév;
std::cin >> vezeteknev >> keresztnév;

std::cout << keresztnév << " " << vezeteknev << std::endl;
```

Írd be egy sorba: `Kiss Anna`. Aztán futtasd újra, és a két szó közé Entert üss. Van különbség?

Próbáld ki - egy egész sor

```
std::string sor;
std::cout << "Írj be egy mondatot: ";
std::getline(std::cin, sor);

std::cout << "Ezt írtad: " << sor << std::endl;
std::cout << "Hossza: " << sor.length() << std::endl;
```

A `std::getline` az Enterig olvas, a szóközökkel együtt.

Kísérlet - a leggyakoribb csapda

Ez a program előbb egy számot, aztán egy egész sort szeretne beolvasni. Próbáld ki!

```
int kor;
std::string nev;

std::cout << "Hány éves vagy? ";
std::cin >> kor;
std::cout << "Mi a teljes neved? ";
std::getline(std::cin, nev);

std::cout << nev << " " << kor << " éves." << std::endl;
```

Volt lehetőség beírni a nevedet? Mit írt ki a program a név helyére?

A magyarázat: a szám után leütött Enter bent maradt a bemenetben. A `std::getline` ezt egy üres sornak látja, és azonnal végez. Írd be ezt a sort a `std::getline` elé, és futtasd újra:

```
std::cin.ignore();
```

Figyelem! Ha `std::cin >>` után `std::getline` következik, mindig tegyél közéjük egy `std::cin.ignore();` sort. Ez átugorja a bent maradt sorvéget.

Próbáld ki - szavak vektorba

A vektorban sztringek is lehetnek:

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> szavak;

    for (int i = 0; i < 3; i++) {
        std::string szo;
        std::cin >> szo;
        szavak.push_back(szo);
    }

    for (int i = 0; i < szavak.size(); i++) {
        std::cout << i + 1 << ". szo: " << szavak[i] << std::endl;
    }
    return 0;
}
```

Írd be a három szót egy sorba, szóközzel elválasztva. Aztán futtasd újra, és írd őket három külön sorba.

Feladatok

2.1. feladat ★☆☆ - Bemutakozás

Kérd be a felhasználó keresztnévét és életkorát. Írd ki így: **Anna 17 éves, jövőre 18 lesz.**

2.2. feladat ★☆☆ - Fordított név

Olvasd be a vezeté- és a keresztnévet (két szó). Írd ki őket fordított sorrendben, vesszővel: **Anna, Kiss.**

2.3. feladat ★☆☆ - Mondat hossza

Olvass be egy egész mondatot, és írd ki, hány karakterből áll.

2.4. feladat ★☆☆ - Kor és név

Javítsd ki a csapdás programot úgy, hogy a teljes nevet is be lehessen írni. Ellenőrizd háromszavas névvel is: **Kiss Anna Maria.**

2.5. feladat ★☆☆ - Bevásárlólista

Kérdezd meg, hány tételből áll a lista. Olvass be ennyi szót egy vektorba, majd írd ki őket sorszámozva.

2.6. feladat ★☆☆ - Többszavas tételek

Alakítsd át az előző programot úgy, hogy egy tétel több szóból is állhasson (például **friss kenyér**). Minden tétel külön sorba kerül. **Tipp:** a darabszám beolvasása után egyszer kell a `std::cin.ignore();`.

3. A sztring karakterei

A sztring karakterek sorozata. Az elemeit ugyanúgy indexeled és járod be, mint egy vektorét.

Próbáld ki

```
std::string szo = "alma";

std::cout << szo[0] << std::endl;
std::cout << szo[3] << std::endl;
```

Sorszám (index)	0	1	2	3
Karakter	a	l	m	a

Tippelj

Mit ír ki ez a két részlet?

```
std::string szo = "informatika";
std::cout << szo[szo.length() - 1] << std::endl;
```

```
std::string szo = "fal";
szo[0] = 'h';
std::cout << szo << std::endl;
```

Egy karakter: a char típus

A sztring egy eleme **char** típusú. Egyetlen karaktert szimpla idézőjelek (aposztrófok) közé írunk: **'a'**.

```
std::string szo = "alma";
char elso = szo[0];

std::cout << elso << std::endl;
```

Kísérletezz: írd át a **szo[0] = 'h'**; sorban az aposztrófokat dupla idézőjelre. Mit mond a fordító?

Figyelem! Az **'a'** egy karakter, az **"a"** egy egyetlen betűből álló sztring. A kettő nem ugyanaz, és nem cserélhető fel.

Próbáld ki - bejárás

```
std::string szo = "informatika";

for (int i = 0; i < szo.length(); i++) {
    std::cout << szo[i] << std::endl;
}
```

Hasonlítsd össze a vektor bejárásával. Mi a különbség?

Próbáld ki - megszámolás

Hány **a** betű van a szóban?

```
std::string szo = "informatika";

int db = 0;
for (int i = 0; i < szo.length(); i++) {
    if (szo[i] == 'a') {
        db++;
    }
}

std::cout << db << std::endl;
```

Feladatok

3.1. feladat ★☆☆ - Első és utolsó

Olvass be egy szót, és írd ki az első és az utolsó betűjét.

3.2. feladat ★☆☆ - Betűnként

Írd ki a beolvasott szó betűit egy sorba, szóközzel elválasztva: **a l m a**.

3.3. feladat ★☆☆ - Visszafelé

Írd ki a beolvasott szót visszafelé: az **alma** szóból **amla** legyen.

3.4. feladat ★☆☆ - Hány e betű?

Olvass be egy egész mondatot. Számold meg, hány **e** betű van benne.

3.5. feladat ★☆☆ - Szóközök

Számold meg, hány szóköz van a beolvasott mondatban. Hány szóból áll a mondat, ha a szavak között mindig pontosan egy szóköz van?

3.6. feladat ★☆☆ - Magánhangzók

Számold meg, hány magánhangzó (a, e, i, o, u) van a beolvasott mondatban.

3.7. feladat ★☆☆ - Aláhúzás

Cseréld ki a beolvasott mondatban minden szóközt aláhúzásjelre (**_**), majd írd ki a mondatot:
a_kutya_ugat.

3.8. feladat ★☆☆ - Van-e benne?

Döntsd el, van-e **z** betű a beolvasott szóban. **Tipp:** **bool** változó, ahogy a vektoros keresésnél.

4. Karakterek vizsgálata és átalakítása

A számítógép minden karaktert egy számként, a kódjával tárol. Ezt kihasználva vizsgálhatod és alakíthatod a betűket.

Próbáld ki - a karakter kódja

```
char c = 'a';
int kod = c;

std::cout << c << " kódja: " << kod << std::endl;
```

Próbáld ki a táblázat többi karakterével is, és írd be, amit kapsz!

Karakter	'a'	'b'	'z'	'A'	'Z'	'0'	'9'	' '
Kódja	97							

Mit veszel észre? Hogyan követik egymást a betűk? Melyik a kisebb: a kisbetűk vagy a nagybetűk kódja?

Tippelj - számolás karakterekkel

```
char c = 'a';
c = c + 1;
std::cout << c << std::endl;

char d = 'A' + 25;
std::cout << d << std::endl;
```

Próbáld ki - milyen karakter ez?

Mivel a betűk kódjai sorban követik egymást, összehasonlítással eldöntheted, mi micsoda:

```
char c = 'k';

if (c >= 'a' && c <= 'z') {
    std::cout << "kisbetu" << std::endl;
}
```

Erre kész függvények is vannak. Kell hozzájuk az `#include <cctype>` sor.

Függvény	Mikor igaz?
<code>std::isalpha(c)</code>	ha <code>c</code> betű
<code>std::isdigit(c)</code>	ha <code>c</code> számjegy
<code>std::islower(c)</code>	ha <code>c</code> kisbetű
<code>std::isupper(c)</code>	ha <code>c</code> nagybetű
<code>std::isspace(c)</code>	ha <code>c</code> szóköz, tabulátor vagy sortörés

Kísérlet - kisbetűből nagybetű

```
char c = 'a';

std::cout << std::toupper(c) << std::endl;
```

Nagy **A** betűt vártál? Mit kaptál helyette? Nézd meg a táblázatodat!

A `std::toupper` a nagybetű **kódját** adja vissza. Ha `char` változóba teszed, újra betű lesz belőle:

```
char nagy = std::toupper(c);
std::cout << nagy << std::endl;
```

A `std::tolower` ugyanígy működik a másik irányba. Így lesz egy egész szóból nagybetűs:

```
std::string szo = "informatika";

for (int i = 0; i < szo.length(); i++) {
    szo[i] = std::toupper(szo[i]);
}

std::cout << szo << std::endl;
```

Próbáld ki - számjegyből szám

```
char c = '7';  
int ertekek = c - '0';  
  
std::cout << ertekek + 1 << std::endl;
```

Miért működik? Nézd meg a táblázatodban a '0' kódját, és számold ki a '7' kódját. Mit ír ki a program, ha a kivonást elhagyod (`int ertekek = c;`)?

Kísérlet - ékezetes betűk

Hány betűből áll a `korte` és a `körte` szó? Mit mond a program?

```
std::string a = "korte";  
std::string b = "körte";  
  
std::cout << a.length() << " " << b.length() << std::endl;
```

Figyelem! Az ékezetes betűket a C++ sok gépen két karakterként tárolja. Ezért a hossz, az indexelés és a `std::toupper` sem működik velük megbízhatóan. Ebben a jegyzetben ékezet nélküli szövegekkel dolgozunk.

Feladatok

4.1. feladat ★☆☆ - Kódtábla

Írd ki az angol ábécé kisbetűit a kódjukkal együtt, soronként egyet: `a 97`. **Tipp:** a ciklusváltozó `char` is lehet: `for (char c = 'a'; c <= 'z'; c++)`.

4.2. feladat ★☆☆ - Csupa nagybetű

Olvass be egy mondatot, és írd ki csupa nagybetűvel.

4.3. feladat ★☆☆ - Számjegyek száma

Olvass be egy sort, amelyben betűk és számok vegyesen vannak (például `Petofi utca 12, 3. emelet 14`). Hány számjegy van benne?

4.4. feladat ★☆☆ - Számjegyek összege

Olvass be egy számot sztringként (például `2026`). Add össze a számjegyeit. A példában az eredmény 10.

4.5. feladat ★☆☆ - Nagy kezdőbetű

Olvass be egy nevet, akárhogyan is van írva (`aNNa`). Írd ki helyesen: az első betű nagy, a többi kicsi (`Anna`).

4.6. feladat ★☆☆ - Karakterstatisztika

Olvass be egy sort. Számold meg külön a kisbetűket, a nagybetűket, a számjegyeket és az összes többi karaktert.

4.7. feladat ★★ - Jelszóellenőrző

Egy jelszó akkor erős, ha legalább 8 karakter hosszú, és van benne kisbetű, nagybetű és számjegy is. Olvass be egy jelszót, és írd ki, erős-e. Ha nem, sorold fel, melyik feltétel nem teljesül.

5. Részsztring és keresés

A `substr` kivág egy darabot a szövegből. A `find` megmondja, hol van benne egy betű vagy egy szövegrész.

Próbáld ki - kivágás

```
std::string szo = "informatika";

std::cout << szo.substr(0, 4) << std::endl;
std::cout << szo.substr(2, 5) << std::endl;
std::cout << szo.substr(6) << std::endl;
```

A táblázat segít kitalálni, mit jelent a két szám:

Sorszám (index)	0	1	2	3	4	5	6	7	8	9	10
Karakter	i	n	f	o	r	m	a	t	i	k	a

A `szo.substr(honnan, hany)` a `honnan` indextől kezdve `hany` darab karaktert ad vissza. Ha csak egy számot adsz meg, a szöveg végéig tart.

Figyelem! A `substr` második száma **darabszám**, nem a vég indexe!

Tippelj - substr-totó

Töltsd ki a középső oszlopot fejből, a fenti táblázat alapján. Utána ellenőrizd géppel. A `szo` értéke végig "informatika".

Kifejezés	A tippem	A program szerint
<code>szo.substr(0, 2)</code>		
<code>szo.substr(4, 3)</code>		
<code>szo.substr(8)</code>		
<code>szo.substr(szo.length() - 3)</code>		
<code>szo.substr(3, 100)</code>		
<code>szo.substr(20)</code>		

Az utolsó két sor meglepő lehet. Mi a különbség a túl nagy darabszám és a túl nagy kezdőindex között?

Próbáld ki - keresés

```
std::string mondat = "a kutya ugat";

std::cout << mondat.find("kutya") << std::endl;
std::cout << mondat.find('u') << std::endl;
std::cout << mondat.find("ugat") << std::endl;
```

A `find` annak az indexét adja meg, ahol a keresett szöveg vagy karakter **először** kezdődik. A találat helyét elteheted egy változóba: `int hely = mondat.find("kutya");`

Kísérlet - ha nincs találat

```
std::cout << mondat.find("macska") << std::endl;
```

Mit kaptál? Ez a furcsa, óriási szám a „nincs találat” jele. Külön neve van: `std::string::npos`. Így kérdezd meg, hogy van-e találat:

```
if (mondat.find("macska") != std::string::npos) {
    std::cout << "Van benne." << std::endl;
} else {
    std::cout << "Nincs benne." << std::endl;
}
```

Próbáld ki - keresés egy adott helytől

A `find` második száma megmondja, honnan induljon a keresés:

```
std::string mondat = "a kutya ugat";

std::cout << mondat.find('a') << std::endl;
std::cout << mondat.find('a', 1) << std::endl;
std::cout << mondat.find('a', 7) << std::endl;
```

Tippelj, aztán futtasd. Hogyan találnád meg egymás után az összes **a** betűt?

Feladatok

5.1. feladat ★☆☆ - Eleje és vége

Olvass be egy legalább hatbetűs szót. Írd ki az első három és az utolsó három karakterét.

5.2. feladat ★☆☆ - Monogram

Olvasd be a vezeté- és a keresztnévet. Írd ki a monogramot pontokkal: **K. A.**

5.3. feladat ★☆☆ - E-mail-cím

Olvass be egy e-mail-címet (például **anna.kiss@iskola.hu**). Írd ki külön a **@** előtti és a **@** utáni részt.

5.4. feladat ★☆☆ - Kiterjesztés

Olvass be egy fájlnevet (például **dolgozat.docx**). Írd ki a kiterjesztését, vagyis a pont utáni részt. Ha nincs benne pont, ezt írd ki: **Nincs kiterjesztes.**

5.5. feladat ★☆☆ - Dátum

Olvass be egy dátumot ebben a formában: **2026.10.04**. Írd ki külön sorba az évet, a hónapot és a napot.

5.6. feladat ★☆☆ - Benne van-e?

Olvass be egy mondatot, majd egy keresett szót. Írd ki, szerepel-e a szó a mondatban, és ha igen, hányadik indexen kezdődik.

5.7. feladat ★★★ - Hányszor?

Számold meg, hányszor fordul elő a keresett szövegrész a mondatban. **Tipp:** ciklusban keress, mindig az előző találat utáni helytől indulva, amíg van találat.

6. Sztring átalakítása és építése

Egy sztringből törölhetsz, beszúrhatsz bele, és kicserélheted egy részét. A leggyakoribb módszer mégis az, hogy karakterenként egy új sztringet építesz.

Tippelj - törlés, beszúrás, csere

Mindhárom kiírás előtt tippelj. Az index-táblázat az előző fejezetben van.

```
std::string s = "informatika";

s.erase(0, 2);
std::cout << s << std::endl;

s.insert(0, "in");
std::cout << s << std::endl;

s.replace(0, 2, "re");
std::cout << s << std::endl;
```

Művelet	Jelentés
<code>s.erase(honnan, hany)</code>	töröl <code>hany</code> karaktert a <code>honnan</code> indextől
<code>s.insert(hova, mit)</code>	beszúrja a <code>mit</code> szöveget a <code>hova</code> index elé
<code>s.replace(honnan, hany, mire)</code>	<code>hany</code> karaktert kicserél a <code>mire</code> szövegre

Próbáld ki - új sztring építése

Így tünteted el a szóközöket egy mondatból:

```
std::string mondat = "a kutya ugat";
std::string uj = "";

for (int i = 0; i < mondat.length(); i++) {
    if (mondat[i] != ' ') {
        uj += mondat[i];
    }
}

std::cout << uj << std::endl;
```

Ismerős? Ez a kiválogatás a vektoros jegyzetből. A `push_back` helyett itt a `+=` fűzi az új karaktert a sztring végére.

Tippelj

Ez a ciklus csak egy apróságban tér el az előzőtől. Mit ír ki?

```
std::string szo = "alma";
std::string uj = "";

for (int i = 0; i < szo.length(); i++) {
    uj = szo[i] + uj;
}

std::cout << uj << std::endl;
```

Mi a különbség az `uj += szo[i];` és az `uj = szo[i] + uj;` között?

Tipp: Az `s.empty()` igaz, ha a sztring üres. Ugyanazt jelenti, mint az `s.length() == 0`.

Feladatok

6.1. feladat ★☆☆ - Egyetlen csere

Az `informatika` szóból egyetlen `replace` hívással csinálj `matematika` szót.

6.2. feladat ★☆☆ - Szóközök nélkül

Olvass be egy mondatot, és írd ki szóközök nélkül.

6.3. feladat ★☆☆ - Megfordítás

Olvass be egy szót. Állítsd elő a megfordítását egy új sztringben, és írd ki mindkettőt.

6.4. feladat ★☆☆ - Palindrom

A palindrom visszafelé olvasva is ugyanaz, például `gorog` vagy `kajak`. Döntsd el a beolvasott szóról, hogy palindrom-e. **Tipp:** két sztringet az `==` jellel hasonlíthatsz össze.

6.5. feladat ★☆☆ - Magánhangzók nélkül

Írd ki a beolvasott szót a magánhangzói nélkül: az `informatika` szóból `nfrmtk` lesz.

6.6. feladat ★☆☆ - Betűcsere

Cseréld ki a beolvasott mondatban minden `a` betűt `e` betűre.

6.7. feladat ★★ - Dadogás

Duplázd meg a beolvasott szó minden betűjét: az `alma` szóból `aallmmaa` lesz.

6.8. feladat ★★ - Cenzúra

Olvass be egy mondatot és egy tiltott szót. A tiltott szó első előfordulását cseréld ki ugyanannyi csillagra, ahány betűből áll. Ha a szó nincs a mondatban, a mondat maradjon változatlan.

7. Összehasonlítás és rendezés

Két sztringet ugyanazokkal a jelekkel hasonlíthatsz össze, mint két számot. Az a „kisebb”, amelyik a szótárban előbb áll.

Próbáld ki

```
std::string a, b;
std::cin >> a >> b;

if (a == b) {
    std::cout << "Egyformak." << std::endl;
} else if (a < b) {
    std::cout << a << " van elobb." << std::endl;
} else {
    std::cout << b << " van elobb." << std::endl;
}
```

Tippelj - melyik van előbb?

Töltsd ki a tippetet, aztán ellenőrizd a fenti programmal.

a	b	Tippem: melyik van előbb?	A program szerint
alma	korte		
alma	almafa		
abc	abd		
Zebra	alma		
alma	Alma		
10	9		

Az utolsó három sor meglepő lehet. A magyarázat a 4. fejezetben van: a program karakterenként, a **kódok** alapján hasonlít. A nagybetűk kódja kisebb, mint a kisbetűké. A **10** és a **9** pedig itt szöveg, nem szám: az **1** kódja kisebb, mint a **9** kódja.

Tipp: Ha a kis- és nagybetűk különbsége nem számít, alakítsd mindkét szót kisbetűssé az összehasonlítás előtt.

Próbáld ki - rendezés

A `std::sort` növekvő sorrendbe rendez egy vektort. Kell hozzá az `#include <algorithm>` sor.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> nevek = {"Dani", "Anna", "Csilla", "Bence"};

    std::sort(nevek.begin(), nevek.end());

    for (int i = 0; i < nevek.size(); i++) {
        std::cout << nevek[i] << std::endl;
    }
    return 0;
}
```

Próbáld ki számokat tároló vektorral is. Működik?

Feladatok

7.1. feladat ★☆☆ - Jelszó kétszer

Kérd be kétszer ugyanazt a jelszót. Írd ki, hogy a két beírás egyezik-e.

7.2. feladat ★☆☆ - Ábécérendben

Olvass be két szót, és írd ki őket ábécérendben.

7.3. feladat ★★☆☆ - Kisbetű, nagybetű

Olvass be két szót. Döntsd el, hogy egyformák-e, ha a kis- és nagybetűk különbségét nem vesszük figyelembe (**Alma** és **alma** egyformának számít).

7.4. feladat ★★☆☆ - Névsor

Kérdezd meg, hány név lesz, olvasd be őket egy vektorba, majd írd ki ábécérendben.

7.5. feladat ★★☆☆ - Leghosszabb szó

Olvass be öt szót egy vektorba. Írd ki a leghosszabbat. **Tipp:** maximumkiválasztás, csak most a szavak hosszát hasonlítsd össze.

7.6. feladat ★★☆☆ - Rendezés nélkül

Olvass be öt szót, és írd ki azt, amelyik az ábécében a legelső. A `std::sort` most nem használható.

8. Szám és szöveg között

Az "123" szöveg és az 123 szám nem ugyanaz. A `std::stoi` szövegből számot, a `std::to_string` számból szöveget csinál.

Tippelj

```
std::string a = "12";
std::string b = "34";

std::cout << a + b << std::endl;
std::cout << std::stoi(a) + std::stoi(b) << std::endl;
```

Függvény	Mit csinál?
<code>std::stoi(s)</code>	szövegből egész szám
<code>std::stod(s)</code>	szövegből tizedes tört (tizedesponnttal)
<code>std::to_string(x)</code>	számból szöveg

Kísérlet - mit bír el a stoi?

Próbáld ki egyesével, és írd be az eredményt. Ha a program hibával leáll, azt is jegyezd fel.

Kifejezés	Eredmény
<code>std::stoi("42")</code>	
<code>std::stoi(" 42")</code>	
<code>std::stoi("42abc")</code>	
<code>std::stoi("3.99")</code>	
<code>std::stoi("abc")</code>	

Figyelem! A `std::stoi` hibával leállítja a programot, ha a szöveg nem számmal kezdődik. Átalakítás előtt gondold végig, biztosan szám van-e a sztringben.

Próbáld ki - számból szöveg

```
int ev = 2026;
std::string s = std::to_string(ev);

std::cout << s.length() << std::endl;
std::cout << s[0] << std::endl;
std::cout << "Az ev: " + s << std::endl;
```

Szöveggént a számnak hossza és karakterei vannak. Próbáld ki a `std::to_string(3.5)` kifejezést is: mit kapsz?

Feladatok

8.1. feladat ★☆☆ - Összeadó

Olvass be két számot **sztringként**. Írd ki az összefűzésüket és az összegüket is.

8.2. feladat ★☆☆ - Hány jegyű?

Olvass be egy egész számot **int** változóba. Írd ki, hány számjegyből áll.

8.3. feladat ★☆☆ - Születési év

Olvass be egy születési dátumot (2009.05.17). Írd ki, hány éves lesz az illető 2026 végén. **Tipp:** vágd ki az évet, és alakítsd számmá.

8.4. feladat ★☆☆ - Óra és perc

Olvass be egy időpontot **ora:perc** formában (07:45). Írd ki, hány perc telt el éjfél óta. A példában 465.

8.5. feladat ★☆☆ - Tükörszám

Olvass be egy egész számot **int** változóba. Döntsd el, hogy visszafelé olvasva is ugyanaz-e (például 12321).

8.6. feladat ★★★ - Időtartam

Olvass be két időpontot **ora:perc** formában, ugyanarról a napról. Írd ki, mennyi idő telt el közöttük, így: 2 ora 35 perc.

9. Sor feldarabolása

A feladatokban az adatok gyakran soronként érkeznek, és egy sorban több adat áll. A **std::stringstream** úgy olvas egy sztringből, ahogy a **std::cin** a billentyűzetről.

Próbáld ki - sor szavakra bontása

```
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::string sor;
    std::getline(std::cin, sor);

    std::stringstream ss(sor);
    std::string szo;
    while (ss >> szo) {
        std::cout << szo << std::endl;
    }
    return 0;
}
```

Mi történik itt?

- **#include <sstream>** – ez kell a **std::stringstream** használatához.
- **std::stringstream ss(sor);** – a sor tartalmát betöltjük az **ss** nevű „belső bemenetbe”.
- **ss >> szo** – ugyanúgy olvas egy szót, mint a **std::cin >> szo**.
- **while (ss >> szo)** – addig ismételi, amíg van még beolvasható szó.

Írj be olyan mondatot is, amelyben két szó között több szóköz van. Zavarja ez a programot?

Próbáld ki - a szavak vektorba kerülnek

```
std::vector<std::string> szavak;

std::stringstream ss(sor);
std::string szo;
while (ss >> szo) {
    szavak.push_back(szo);
}

std::cout << "Szavak szama: " << szavak.size() << std::endl;
```

Innentől a szavakkal mindent megtehetsz, amit egy vektor elemeivel.

Próbáld ki - vegyes adatok egy sorban

Egy sorban szöveg és szám is lehet. A `>>` rögtön a megfelelő típusra alakítja:

```
std::string sor = "Anna 17 4.5";
std::stringstream ss(sor);

std::string nev;
int kor;
double atlag;
ss >> nev >> kor >> atlag;

std::cout << nev << " jövöre " << kor + 1 << " éves lesz." << std::endl;
```

Próbáld ki - más elválasztójel

Ha az adatokat nem szóköz, hanem például pontosvessző választja el, a `std::getline` harmadik paramétere segít:

```
std::string sor = "Kiss Anna;17;4.5";
std::stringstream ss(sor);

std::string nev, korSzoveg, atlagSzoveg;
std::getline(ss, nev, ';');
std::getline(ss, korSzoveg, ';');
std::getline(ss, atlagSzoveg, ';');

int kor = std::stoi(korSzoveg);
double atlag = std::stod(atlagSzoveg);

std::cout << nev << " (" << kor << ") atlaga: " << atlag << std::endl;
```

Itt minden darab sztringként érkezik, ezért a számokat a `std::stoi` és a `std::stod` alakítja át. Miért nem lett volna jó itt az `ss >> nev`?

Ha nem tudod előre, hány darab lesz, ciklussal is olvashatsz:

```
std::string resz;
while (std::getline(ss, resz, ';')) {
    std::cout << resz << std::endl;
}
```

Feladatok

9.1. feladat ★☆☆ - Hány szó?

Olvass be egy mondatot, és írd ki, hány szóból áll. Próbáld ki úgy is, hogy két szó közé több szóközt írsz. Hasonlítsd össze a 3.5. feladat megoldásával: melyik számol jól?

9.2. feladat ★☆☆ - Szavak és hosszuk

Írd ki a beolvasott mondat szavait külön sorokba, mindegyik mellé a hosszát: `kutya 5`.

9.3. feladat ★☆☆ - Leghosszabb szó a mondatban

Írd ki a beolvasott mondat leghosszabb szavát.

9.4. feladat ★☆☆ - Fordított szórend

Írd ki a mondat szavait fordított sorrendben: `a kutya ugat` helyett `ugat kutya a`.

9.5. feladat ★☆☆ - Számok egy sorban

Olvass be egy sort, amelyben egész számok állnak szóközzel elválasztva (`3 8 12 5`). Nem tudod előre, hány szám lesz. Írd ki az összegüket és az átlagukat. **Tipp:** a `while (ss >> szam)` ciklus `int` változóval is működik.

9.6. feladat ★☆☆ - Hosszú monogram

Olvass be egy teljes nevet, amely akárhány szóból állhat (`Kiss Anna Maria`). Írd ki a monogramját: `KAM`.

9.7. feladat ★☆☆ - Adatsor

Olvass be egy sort ebben a formában: `Kiss Anna;17;4.5`. Írd ki, hány éves lesz a tanuló jövőre, és hogy az átlaga eléri-e a 4.0-t.

9.8. feladat ★★ - Pontverseny

Előbb olvasd be, hány versenyző van. Utána minden versenyzőről egy sor érkezik `nev;pontszam` formában (`Nagy Bence;87`). Írd ki a legtöbb pontot elérő versenyző nevét és az átlagpontszámot. **Tipp:** ne felejtse el a `std::cin.ignore();` sort a darabszám után.

10. Kihívások

Ezekhez nincs új tudnivaló, csak össze kell raknod, amit eddig tanultál. Válaszd azt, amelyik megtetszik.

10.1. feladat ★☆☆ - Mondatpalindrom

Az `Indul a gorog aludni` mondat visszafelé is ugyanaz, ha a szóközöket és a kis- és nagybetűk különbségét nem nézzük. Írj programot, amely egy beolvasott mondatról ezt eldönti.

10.2. feladat ★☆☆ - Nagy kezdőbetűk

Olvass be egy kisbetűvel írt teljes nevet (`kiss anna maria`). Írd ki úgy, hogy minden szó nagybetűvel kezdődik.

10.3. feladat ★★ - Caesar-rejtjel

Olvass be egy kisbetűs szót. Told el minden betűjét hárommal az ábécében: `a` helyett `d`, `b` helyett `e`. Az ábécé végén körbe kell fordulni: `x` helyett `a`, `y` helyett `b`, `z` helyett `c`. Írd meg a visszafejtést is!

10.4. feladat ★★ - Betűstatisztika

Olvass be egy kisbetűs mondatot. Számold meg, melyik betű hányszor fordul elő, és írd ki a leggyakoribbat. **Tipp:** használj 26 elemű számláló vektort; a `c - 'a'` megadja, hányadik betű a `c` az ábécében.

10.5. feladat ★★ - Rendszám

Egy régi típusú rendszám így néz ki: `ABC-123`. Három nagybetű, kötőjel, három számjegy. Olvass be egy szöveget, és dönts el, megfelel-e ennek a formának.

10.6. feladat ★★ - Anagramma

Két szó anagramma, ha ugyanazokból a betűkből állnak, csak más sorrendben (`roka` és `kora`). Olvass be két szót, és dönts el, anagrammák-e. **Tipp:** a `std::sort` sztring betűit is rendezi:

```
std::sort(s.begin(), s.end());
```

10.7. feladat ★★★ - Tömörítés

Olvass be egy szót, amelyben sok az ismétlődő betű (aaabccdddd). Írd ki tömörítve: minden betű után az áll, hányszor ismétlődik egymás után (a3b1c2d4).

Összefoglaló táblázat (puska)

Szükséges fejlécek: `<string>`, a karakterfüggvényekhez `<cctype>`, a daraboláshoz `<sstream>`, a rendezéshez `<algorithm>`.

Művelet	Jelentés
<code>s.length()</code>	karakterek száma
<code>s[i]</code>	az <code>i</code> indexű karakter (<code>char</code>)
<code>a + b</code> és <code>s += x</code>	összefűzés
<code>std::cin >> s</code>	egy szó beolvasása
<code>std::getline(std::cin, s)</code>	egy egész sor beolvasása
<code>s.substr(honnan, hany)</code>	részsztring kivágása
<code>s.find(mit)</code>	az első előfordulás indexe, vagy <code>std::string::npos</code>
<code>s.erase(honnan, hany)</code>	törlés
<code>s.insert(hova, mit)</code>	beszúrás
<code>s.replace(honnan, hany, mire)</code>	csere
<code>s.empty()</code>	igaz, ha a sztring üres
<code>a == b</code> és <code>a < b</code>	összehasonlítás a karakterkódok szerint
<code>std::toupper(c)</code> és <code>std::tolower(c)</code>	nagybetűs, illetve kisbetűs változat kódja
<code>std::isdigit(c)</code> és <code>std::isalpha(c)</code>	számjegy-e, betű-e
<code>c - '0'</code>	számjegy karakter értéke számként
<code>std::stoi(s)</code> és <code>std::stod(s)</code>	szövegből szám
<code>std::to_string(x)</code>	számból szöveg
<code>std::stringstream ss(sor);</code>	sor előkészítése a daraboláshoz
<code>ss >> szo</code>	következő szó (vagy szám) a sorból
<code>std::getline(ss, resz, ';')</code>	következő darab a megadott elválasztójelig

Figyelem! Három dolog, amit soha ne felejts el: a `std::cin >>` után a `std::getline` elé kell a `std::cin.ignore();`, az `'a'` karakter és az `"a"` sztring nem ugyanaz, és a `find` a sikertelen keresést a `std::string::npos` értékkel jelzi.